

■ Learn About RNNs & Curate Colab Explanations

Shilpa Yelkur Ramakrishnaiah

Student ID: 019151782

Course: Deep Learning — CMPE 258

Assignment Due: May 10, 2025 by 11:59 PM | Points: 300

GitHub Repository

github.com/ShilpaYR/Learn-about-RNNs-and-curate-colab-explanations

Notebooks Covered

RNN / LSTM / GRU / WaveNet

10 Years of Deep NLP

Vision Transformers (ViT, CLIP, DINOv2, SAM)

GNN Fundamentals

■ Assignment Overview & Student Information

Assignment	Learn About RNNs and Curate Colab Explanations
Student Name	Shilpa Yelkur Ramakrishnaiah
Student ID	019151782
Course	Deep Learning — CMPE 258, Spring 2025
Due Date	May 10, 2025 by 11:59 PM
Points	300
Submission	Website URL (GitHub Repository)
GitHub Repo	https://github.com/ShilpaYR/Learn-about-RNNs-and-curate-colab-explanations

This assignment involved executing four Google Colab notebooks on deep learning topics provided by the instructor. Each notebook was copied into Google Drive and GitHub with world-sharing permissions enabled. The notebooks were fully executed with all outputs visible and then archived in the GitHub repository. For each notebook, one YouTube video was recorded explaining every code block and its output, code-block-by-code-block. The README in the repository contains a structured table linking each notebook to its corresponding YouTube video.

■ Deliverables — Notebooks & YouTube Videos

#	Notebook File	Topic	YouTube Link
1	<code>final_rnn_lstm_gru_wavenet_zero_to_hero.ipynb</code>	Sequence Models: RNN → LSTM → GRU → WaveNet	https://youtu.be/LgypN2QRZ8o
2	<code>final_nlp_deep_learning_10_years_tutorial.ipynb</code>	10 Years of Deep Learning in NLP	https://youtu.be/U5kaGbb4624
3	<code>final_vision_transformers_tutorial.ipynb</code>	Vision Transformers: ViT, CLIP, DINOv2, SAM	https://youtu.be/eia2zWUUgOY
4	<code>final_gnn_fundamentals_tutorial.ipynb</code>	GNN Fundamentals: Zero to Hero	https://youtu.be/8IRX_lkbWWU

Notebook 1 — Sequence Models: RNN → LSTM → GRU → WaveNet

YouTube:

<https://youtu.be/LgypN2QRZ8o>

Inspired by Aurélien Géron's *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (O'Reilly), this notebook builds a complete understanding of sequence models in PyTorch, from vanilla RNNs all the way to WaveNet. A single consistent benchmark — **character-level language modeling** — is used across all four architectures so their performance can be compared directly and fairly. The task: given a window of characters, predict the next character. Models are trained on a self-contained prose corpus (~2,200 characters) and text generation is evaluated at temperatures of 0.2, 0.8, and 1.5 to show how randomness affects output coherence.

Key Topics & Concepts	PyTorch Classes & Code
• Why sequential order matters (text, audio, time-series, DNA)	• torch.nn.RNN (batch_first=True)
• Sliding-window CharDataset with PyTorch DataLoader	• CharDataset(Dataset) with sliding windows
• Shared train/val split (90/10) for fair comparison	• train_model() with Adam optimizer
• Vanilla RNN: $h_t = \tanh(W_{ih} \cdot x_t + W_{hh} \cdot h_{t-1})$	• torch.nn.utils.clip_grad_norm_()
• Vanishing gradient problem — tanh squash at every step	• torch.nn.LSTM — (h_n, c_n) dual state
• LSTM: forget gate (f), input gate (i), output gate (o), cell state (c)	• CharLSTM: LSTM + nn.Embedding + nn.Linear
• Cell-state highway: additive update prevents gradient decay	• CharGRU: GRU + nn.Embedding + nn.Linear
• GRU: update gate (z) and reset gate (r), no separate cell state	• DeepCharLSTM: num_layers=2, dropout=0.2
• Deep LSTM (2 layers, dropout=0.2 between layers)	• generate_text() with multinomial sampling
• Gradient clipping: clip_grad_norm_(params, 1.0)	• F.softmax with temperature scaling
• Temperature parameter for text generation (0.2 / 0.8 / 1.5)	• CausalConv1dBlock with F.pad (left only)
• WaveNet: causal & dilated 1-D convolutions, dilation = 2^i	• CharWaveNet: nn.ModuleList of conv blocks
• Receptive field = $2^{\text{num_layers}}$ (128 chars with 7 layers)	• Residual connections in each WaveNet block
• Grand comparison: loss, time/epoch, parameter count	• Comprehensive benchmark summary table

Architecture Comparison (from Notebook)

Architecture	Year	Gates / State	Key Advantage	Key Limitation
Vanilla RNN	1986	None / hidden state h	Simple, few params	Vanishing gradients, short memory
LSTM	1997	3 gates + cell state c	Long-range memory via additive updates	4× parameters vs RNN
GRU	2014	2 gates (z, r), merged state	~25% fewer params than LSTM, comparable quality	May lag on very long sequences

WaveNet	2016	No gates — dilated conv	Fully parallel training on GPU	Slow autoregressive generation
---------	------	-------------------------	--------------------------------	--------------------------------

Training Infrastructure (shared across all models)

The notebook defines a reusable **train_model()** function that handles the full PyTorch training loop: forward pass, cross-entropy loss computation, **clip_grad_norm_** for exploding-gradient prevention, Adam optimizer step, and per-epoch tracking of train/val loss and wall-clock time. A companion **generate_text()** function samples from any trained model character-by-character, using a context window of SEQ_LENGTH=64 and a configurable temperature parameter. Hyperparameters shared across all models: EMBED_DIM=64, HIDDEN_DIM=128, BATCH_SIZE=64, NUM_EPOCHS=20, LR=0.003. This consistent setup ensures every architecture is evaluated on identical conditions.

WaveNet — Dilated Causal Convolutions

The WaveNet section introduces a fundamentally different approach: rather than recurrence, causal dilated 1-D convolutions are stacked so that **dilation doubles at each layer** (1, 2, 4, 8, 16, 32, 64). With 7 layers the receptive field reaches $2^7 = 128$ characters — enough to capture rich context. Each **CausalConv1dBlock** pads the input on the left only (padding = (kernel-1)×dilation) to ensure no future information leaks, applies LayerNorm and GELU activation, then adds a residual connection. Because all positions are computed in parallel during training (unlike sequential RNNs), WaveNet is significantly faster to train on GPU. The notebook concludes with a full benchmark table: loss, training time per epoch, and parameter count for all five variants (RNN, LSTM, GRU, Deep LSTM, WaveNet).

Notebook 2 — 10 Years of Deep Learning in NLP

YouTube:

<https://youtu.be/U5kaGbb4624>

This notebook provides a visual, beginner-friendly journey through 50 key concepts that transformed Natural Language Processing from 2013 to the era of ChatGPT. The narrative arc spans five chapters — from basic tokenization and word embeddings through sequential RNN/LSTM models, the invention of attention, the full Transformer architecture, large pre-trained language models (BERT, GPT, T5, XLNet), and finally human alignment techniques such as RLHF that produced ChatGPT and GPT-4. Every concept is accompanied by hand-written Python code, visual diagrams, and interactive demonstrations.

Chapter-by-Chapter Topics	Key Code / Implementations
• Chapter 1: Tokenization — character, word, subword (BPE)	• <code>visualize_tokenization()</code> — 3-method side-by-side plot
• SimpleTokenizer built from scratch with [PAD],[UNK],[START],[END]	• SimpleTokenizer: <code>fit()</code>, <code>encode()</code>, <code>decode()</code>
• Word embeddings: dense 4-D vectors capturing semantic meaning	• WordEmbeddings class with cosine similarity
• Cosine similarity and word similarity heatmap	• <code>visualize_word_similarities()</code> — seaborn heatmap
• Word analogy: King – Man + Woman $\approx$ Queen	• <code>word_analogy()</code> — $A - B + C$ vector arithmetic
• PCA projection of embeddings to 2-D	• <code>visualize_embeddings_2d()</code> using sklearn PCA
• Chapter 2: RNN hidden state recurrence, $h_t = \tanh(W \cdot [h_{t-1}, x_t])$	• SimpleRNN.forward() processing sequence step-by-step
• LSTM three gates (forget, input, output) and cell state highway	• LSTM gate equations implemented in NumPy
• GRU update and reset gates — simplification of LSTM	• GRU simplified 2-gate implementation
• Encoder-Decoder Seq2Seq for translation tasks	• BahdanauAttention — alignment score computation
• Bahdanau attention mechanism (2015)	• <code>scaled_dot_product_attention(Q, K, V)</code> in PyTorch
• Chapter 3: Transformer self-attention Q, K, V formulation	• MultiHeadAttention with num_heads projections
• Multi-head attention, positional encoding, FFN sublayers	• PositionalEncoding using sin/cos functions
• Full Transformer encoder-decoder architecture	• TransformerBlock — attention + FFN + LayerNorm + residual
• Chapter 4: BERT (encoder-only, masked LM, NSP)	• BERTTokenizer demonstration with [MASK] tokens
• GPT (decoder-only, causal LM, autoregressive)	• GPT-style causal mask (lower triangular)
• XLNet permutation language model; T5 text-to-text	• LoRALayer — low-rank weight decomposition demo
• LoRA: low-rank parameter-efficient fine-tuning	• RLHFRewardModel — preference scoring
• Knowledge distillation: teacher-student compression	• Visualization of alignment pipeline stages
• Chapter 5: Hallucination problem in LLMs	•
• RLHF: reward model + PPO optimization	•

NLP Timeline — Evolution of Key Architectures

Year	Model	Key Innovation	Notebook Coverage
2013	Word2Vec	Dense word embeddings learned from context	Embedding class, cosine similarity, analogy demo
2014	RNN Seq2Seq	Encoder-decoder for sequence translation	SimpleRNN from scratch, hidden state animation
2015	Attention	Alignment scores remove bottleneck in Seq2Seq	BahdanauAttention implementation
2017	Transformer	Self-attention replaces recurrence entirely	Full Q/K/V attention, multi-head, positional encoding
2018	BERT	Bidirectional encoder, masked LM pre-training	Encoder-only architecture, [MASK] token explanation
2019	GPT-2	Large-scale causal language model	Decoder-only architecture, causal masking
2022–23	ChatGPT/GPT-4	RLHF alignment, instruction following	Reward model, PPO training, alignment pipeline

Tokenization Detail

The notebook opens by explaining why computers must convert text to numbers before any neural network can process it. Three tokenization levels are visualized side-by-side: **character-level** (every individual character becomes a token — very long sequences), **word-level** (each word is a token — simple but fails on rare or misspelled words), and **subword-level** (BPE-style splitting of common suffixes — the modern standard used by BERT and GPT). The built-from-scratch **SimpleTokenizer** demonstrates the full pipeline: building a vocabulary from training sentences, encoding text to integer IDs, and decoding back to readable strings — all with step-by-step print output.

Word Embeddings & Analogy Test

A custom **WordEmbeddings** class stores 4-dimensional vectors for twelve words (royalty, common people, animals). Cosine similarity is computed pairwise and displayed as a color-coded heatmap so the learner can visually verify that 'cat' and 'kitten' score near 1.0 while 'king' and 'puppy' score near 0. The famous vector arithmetic **King – Man + Woman ≈ Queen** is reproduced and printed step-by-step. Finally, PCA projects all twelve embedding vectors to 2-D with annotated scatter-plot groupings (royalty in gold, people in teal, animals in coral) and directional arrows connecting gender analogues (king→queen, prince→princess).

Notebook 3 — Vision Transformers — ViT, CLIP, DINOv2, SAM

YouTube:

<https://youtu.be/eia2zWUUgOY>

This notebook documents the paradigm shift that occurred when Transformers — originally designed for NLP — proved they could match or beat CNNs at image recognition. Starting from first-principles self-attention mathematics, the notebook constructs a full **Vision Transformer (ViT)** in PyTorch, then explores three landmark extensions: CLIP (vision-language contrastive pre-training), DINOv2 (self-supervised learning with emergent segmentation), and SAM (Segment Anything Model). The notebook also benchmarks hybrid architectures combining convolutional inductive bias with Transformer global attention (Swin Transformer, ConvNeXt).

Chapter-by-Chapter Topics	PyTorch Implementations
• Chapter 1: Scaled dot-product attention formula: $\text{softmax}(QK^T/d_k) \cdot V$	• <code>scaled_dot_product_attention(Q, K, V, mask)</code> function
• Q=Query ('what am I looking for?'), K=Key ('what do I have?'), V=Value	• <code>MultiHeadAttention(nn.Module)</code> with <code>W_q</code>, <code>W_k</code>, <code>W_v</code>, <code>W_o</code> projections
• Multi-head attention — <code>h</code> parallel heads with <code>d_model/h</code> dimensions each	• <code>visualize_attention()</code> — 2×4 grid of head attention heatmaps
• Attention weight visualization: 8-head heatmap grid	• <code>PatchEmbedding(img_size=224, patch_size=16, embed_dim=768)</code>
• Chapter 2: ViT — treating images as sequences of 16×16 patches	• <code>VisionTransformer(nn.Module)</code> full class
• <code>PatchEmbedding</code> via <code>Conv2d(kernel=16, stride=16)</code> for non-overlapping tiles	• <code>cls_token</code> as <code>nn.Parameter(torch.zeros(1,1,d_model))</code>
• Learnable [CLS] token prepended to patch sequence	• <code>pos_embed</code> as <code>nn.Parameter(torch.zeros(1,num_patches+1,d_model))</code>
• Learnable position embeddings added to patch embeddings	• <code>TransformerEncoderBlock(d_model, num_heads, mlp_ratio)</code>
• <code>TransformerEncoderBlock</code>: Pre-Norm, MHA + FFN + residuals	• <code>nn.Sequential</code> of 12 encoder blocks
• MLP head from [CLS] token to <code>num_classes</code> for classification	• LayerNorm + GELU in MLP sublayer
• ViT model variants: ViT-B/16 (86M params), ViT-L/16 (307M params)	• CLIP <code>DualEncoder</code> conceptual implementation
• Chapter 3: CLIP — dual-encoder image + text contrastive learning	• <code>contrastive_loss()</code> for image-text alignment
• Zero-shot classification: compare image embedding to text descriptions	• <code>zero_shot_classify()</code> using cosine similarity
• Chapter 4: DINOv2 — self-supervised ViT with emerging segmentation	• DINOv2 feature extractor demonstration
• Chapter 5: SAM — promptable segmentation (points, boxes, masks)	• SAM image encoder + prompt encoder conceptual demo

• Chapter 6: Swin Transformer — shifted window local attention	• Swin window partition and shifted attention
• ConvNeXt — modernized ResNet with Transformer design choices	• ConvNeXt block with depthwise conv + LayerNorm

Vision Transformer Model Timeline

Year	Model	Innovation	Impact
2020	ViT	Pure Transformer for image recognition, 16×16 patch embedding	Proved Transformers can match CNNs on ImageNet at scale
2021	CLIP	Contrastive pre-training on 400M image-text pairs	Zero-shot image classification; foundation of multimodal AI
2021	DINO	Self-distillation without labels	Emergent segmentation from features; no annotation needed
2023	DINOv2	Scaled self-supervised ViT on curated data	Best general-purpose visual features across tasks
2023	SAM	Promptable segmentation with points/boxes/masks	Universal segmentation; foundation model for vision
2021	Swin Transformer	Shifted window local attention + hierarchical features	Efficient for dense prediction; won ICCV 2021 Best Paper

Attention Mechanism — Built From Scratch

Chapter 1 implements **scaled_dot_product_attention(Q, K, V, mask)** following the Vaswani et al. formula: $\text{scores} = QK^T/\sqrt{d_k}$, then softmax, then weighted sum of V. The code verifies that each attention weight row sums to 1 (a valid probability distribution). The **MultiHeadAttention** class uses four separate nn.Linear projections (W_q, W_k, W_v, W_o), reshapes tensors to (batch, num_heads, seq_len, d_k) for parallelism, and concatenates all head outputs before the final projection. A visualization function plots all 8 attention head heatmaps simultaneously, illustrating how different heads learn complementary relational patterns across the sequence.

ViT — Image Patchification

The key insight of ViT is that a 224×224 image divided into 16×16 patches produces 196 non-overlapping tokens — exactly analogous to word tokens in NLP. **PatchEmbedding** uses a Conv2d layer with `kernel_size=patch_size` and `stride=patch_size` to efficiently compute patch embeddings in one pass (equivalent to flattening each 16×16×3 patch and applying a linear projection to `embed_dim=768`). The [CLS] token is a learnable vector prepended to the 196 patch tokens; after 12 Transformer encoder blocks this token aggregates global image information and feeds into a linear classification head. A **visualize_patches()** function overlays the 14×14 grid on a synthetic gradient image and displays the first 16 extracted patches side-by-side.

Notebook 4 — Graph Neural Networks — Fundamentals: Zero to Hero

YouTube:

https://youtu.be/8IRX_lkbWWU

Part 1 of a three-part GNN tutorial series. This notebook builds the learner's understanding of graph-structured data from first principles and then derives and implements Graph Convolutional Networks (GCN) in both NumPy and PyTorch. The central thesis is that graphs — which model arbitrary pairwise relationships — are ubiquitous in real-world problems (social networks, molecules, citation graphs, protein structures, knowledge graphs, transportation networks) yet cannot be handled by standard CNNs or MLPs because they lack a fixed grid structure. The solution is the **message passing paradigm**, where each node iteratively aggregates feature information from its neighborhood.

Chapter-by-Chapter Topics	Code & PyTorch Implementations
Chapter 1: Graph definition $G = (V, E)$ — nodes, edges, features	<code>nx.Graph()</code> construction from edge lists
Undirected, directed, weighted, bipartite graph types	<code>nx.spring_layout()</code> with <code>seed=42</code>
Real-world graph applications table (8 domains)	<code>nx.draw()</code> with <code>node_color</code>, <code>edge_color</code>, <code>labels</code>
NetworkX graph construction and spring-layout visualization	<code>visualize_graph_example()</code> — 4-panel: undirected/directed/weighted/bipartite
Chapter 2: Adjacency matrix A — $A_{ij} = 1$ if edge (i,j) exists	<code>demonstrate_adjacency_matrix()</code> — graph + heatmap side by side
Symmetric property of undirected adjacency matrices	<code>np.allclose(A, A.T)</code> — symmetry verification
Edge list representation — sparse, memory-efficient	Degree matrix D: <code>np.diag(A.sum(axis=1))</code>
Adjacency list (dict) — fast neighbor lookup	Normalized Laplacian: $D^{-0.5} @ A @ D^{-0.5}$
Comparison: when each representation excels	GCN forward pass in NumPy: <code>A_hat @ H @ W</code>
Chapter 3: Why CNNs and MLPs fail on graphs (permutation variance)	ReLU activation: <code>np.maximum(0, x)</code>
The message passing paradigm — aggregate, update, repeat	Softmax output: <code>exp(x)/sum(exp(x))</code>
Chapter 4: GCN math derivation — normalized Laplacian	<code>GCNLayer(nn.Module)</code> in PyTorch
Degree matrix D, normalized adjacency $\tilde{A} = D^{-1/2} A D^{-1/2}$	<code>TwoLayerGCN</code> — full classification model
Adding self-loops: $\tilde{A} = A + I$ for self-connections	Node feature matrix X initialization
GCN layer: $H^{[l+1]} = \sigma(\tilde{A} \cdot H^{[l]} \cdot W^{[l]})$	Training loop: Adam + CrossEntropyLoss
2-layer GCN for node classification: <code>softmax($\tilde{A} \cdot \text{ReLU}(\tilde{A} \cdot X \cdot W_0) \cdot W_1$)</code>	Accuracy evaluation on test nodes
Demo: node classification on citation dataset	

Graph Applications Covered in Notebook

Domain	Nodes	Edges	Example Task
--------	-------	-------	--------------

Social Networks	Users	Friendships	Friend recommendation
Molecules	Atoms	Chemical bonds	Drug discovery
Citation Networks	Papers	Citations	Paper classification (Cora demo)
Knowledge Graphs	Entities	Relations	Question answering
Transportation	Locations	Roads	Route optimization
Protein Structures	Amino acids	Interactions	Function prediction
Computer Networks	Devices	Connections	Anomaly detection
Recommender Systems	Users + Items	Interactions	Product recommendation

GCN Math & Implementation

The notebook derives GCN from spectral graph theory. Starting from the raw adjacency matrix A , it computes the **degree matrix D** (diagonal, D_{ii} = number of neighbors) and then the **symmetrically normalized adjacency** $\tilde{A} = D^{-1/2}AD^{-1/2}$. Self-loops are added ($\tilde{A} = A+I$ before normalization) so each node also considers its own features. A single GCN layer computes: $H^{l+1} = \sigma(\tilde{A} \cdot H^l \cdot W^l)$, where W is a learnable weight matrix. The notebook first implements this in **pure NumPy** to expose every matrix multiplication explicitly, then ports to a **PyTorch nn.Module** for scalability. A full 2-layer GCN node classification demo on a citation dataset closes the notebook, with a training loop that prints accuracy at each epoch.

■ Key Learnings & Takeaways

Sequence Modeling with RNNs

Understood why order is fundamental in sequences and built four architectures in PyTorch. The Vanilla RNN introduces recurrence but suffers vanishing gradients because $\tanh$ squashes every gradient. LSTM solves this by introducing a cell-state highway updated additively ($c_t = f \odot c_{t-1} + i \odot c_t$) so gradients can flow unchanged when the forget gate is ~ 1 . GRU achieves similar performance with fewer parameters by merging the forget/input gates into a single update gate. WaveNet demonstrates that dilated causal convolutions with exponentially growing receptive fields can replace recurrence entirely, enabling fully parallel GPU training.

10 Years of Deep NLP

Traced the evolution from Word2Vec embeddings (2013) through RNN Seq2Seq, Bahdanau attention, the Transformer (2017), BERT and GPT large pre-trained models, parameter-efficient fine-tuning via LoRA, and finally human alignment through RLHF that produced ChatGPT. Implemented the full pipeline from tokenization to analogy arithmetic to the Transformer architecture from scratch.

Vision Transformers

Learned how images can be treated as sequences of 16×16 -pixel patches and processed by a standard Transformer encoder. Built Multi-Head Attention and the full ViT in PyTorch. Understood CLIP's contrastive dual-encoder for zero-shot classification, DINOv2's self-supervised emergent segmentation, and SAM's promptable universal segmentation. Also explored hybrid CNN-Transformer architectures (Swin, ConvNeXt).

Graph Neural Networks

Mastered graph data structures (adjacency matrix, edge list, adjacency list) and the message passing paradigm where nodes iteratively aggregate neighbor features. Derived GCN from the normalized graph Laplacian and implemented it in both NumPy (for full mathematical transparency) and PyTorch (for scalability). Demonstrated node classification on a citation network with a 2-layer GCN.

■ ■ GitHub Repository

<https://github.com/ShilpaYR/Learn-about-RNNs-and-curate-colab-explanations>

Repository Structure:

```
Learn-about-RNNs-and-curate-colab-explanations/  
  ■ ■ ■ README.md – main index with Colab + YouTube table  
  ■ ■ ■ notebooks/  
    ■ ■ ■ final_rnn_lstm_gru_wavenet_zero_to_hero.ipynb  
    ■ ■ ■ final_nlp_deep_learning_10_years_tutorial.ipynb  
    ■ ■ ■ final_vision_transformers_tutorial.ipynb  
    ■ ■ ■ final_gnn_fundamentals_tutorial.ipynb
```

■ Assignment Checklist

■	All 4 notebooks copied to Google Drive with world-sharing permissions
■	All 4 notebooks fully executed with all outputs visible
■	All 4 notebooks archived in GitHub repository
■	1 YouTube video recorded per notebook — code block by code block
■	README contains structured table of Colab links + YouTube links
■	GitHub repository publicly accessible with proper folder organization